

MIRA MODERNIZE

Generic Application Modernization Platform

Solution Architecture & Functional Workflow

Document	Details
Purpose	Define the proposed product workflow, agent responsibilities and architecture.
Audience	Engineering, Architecture, Innovation Labs, Delivery and Client Technology teams.
Primary Capability	Assess and execute application modernization using AI agents with controlled human approval.
Initial Transformation Example	Java 8 → Java 17; the platform is not restricted to this scenario.

Design Principle

The platform is generic. A modernization request may represent a version upgrade, framework/library upgrade, technology replacement, architecture change, or a combination of these. The request defines the transformation profile; the platform orchestrates the appropriate assessment, planning, execution and validation agents.

1. Purpose and Objective

Mira Modernize is an AI-driven platform for analyzing and executing changes to existing software applications. Its purpose is to reduce the manual effort involved in understanding an application, determining the impact of a requested modernization, planning the required changes, implementing those changes, and validating the resulting application.

The platform must support four broad categories of modernization:

- Version modernization — for example, upgrading a runtime or language version.
- Framework or library modernization — for example, replacing or upgrading an application framework.
- Technology transformation — for example, moving an application from one technology/framework to another.
- Application enhancement — for example, introducing a new capability into an existing application while preserving existing behavior.

2. Scope

The platform is organized around two primary business capabilities:

Capability	Purpose
Assessment	Understand the application, analyze the requested change, identify impact and determine a feasible modernization approach.
Migration / Transformation	Execute the approved modernization approach, validate the result and produce evidence of the changes and outcome.

The platform is not tied to a specific programming language, cloud provider, repository provider or AI model.

3. Core Concepts

Concept	Description	Example
Source	Current application state.	Java 8 + Struts application
Target	Desired application state.	Java 17 + supported web framework
Transformation	The requested change from source to target.	Java 8 → Java 17
Modernization Profile	Structured definition of how a particular transformation should be assessed/executed.	Java runtime upgrade profile
Finding	An observed issue or impact supported by repository evidence.	Dependency is incompatible with target runtime
Decision	An approved strategy selected by the user.	Upgrade dependency and migrate affected framework
Migration Task	An executable unit derived from the approved plan.	Update build configuration and compile with JDK 17
Validation	Evidence that the resulting application satisfies agreed checks.	Build succeeds and regression tests pass

4. End-to-End Functional Workflow

The workflow is deliberately linear at the business level, while individual agent activities within a stage may run in parallel.

Stage	System Behavior
1. Create Modernization Request	User supplies repository, source state, target state and any known constraints.
2. Discover Application	Agents inspect repository structure, technologies, dependencies, configuration and tests.
3. Assess Impact	Agents compare the current application with the requested target and produce evidence-based findings.
4. Determine Feasibility	The platform consolidates findings and identifies blockers, risks and possible approaches.
5. Human Decision	User reviews the assessment and approves a modernization strategy or requests reassessment.
6. Generate Migration Plan	The approved strategy is converted into ordered phases and executable tasks.
7. Execute Migration	Specialized agents modify the application in an isolated workspace/branch.
8. Validate Iteratively	Builds and tests are executed; failures are analyzed and remediation is attempted.
9. Complete / Escalate	The platform produces a final report or pauses when an unresolved decision requires human input.

5. Stage A — Modernization Request

A modernization request should be represented as a structured object rather than a free-form prompt.

Input	Description
Repository	Repository location and access details.
Source	Current language/technology/framework/version.
Target	Desired language/technology/framework/version.
Change Type	Version upgrade, framework upgrade, technology switch, enhancement, or combination.
Scope	Entire repository, selected modules, or selected components.
Constraints	Security, compatibility, coding standards, deployment or client-specific constraints.
Validation Requirements	Tests/checks expected after migration.

Example

A request could be: 'Analyze the existing Java 8 application and determine the work required to run it on Java 17. If the current web framework is not suitable for the target state, identify viable alternatives and present them for approval.'

6. Stage B — Assessment

Assessment is the intelligence-gathering stage. Its output is not code; its output is a reliable understanding of the application and the impact of the requested transformation.

Agent	Responsibility	Primary Output
-------	----------------	----------------

Repository Discovery Agent	Build repository inventory and identify application boundaries.	Repository structure, modules, source/config/test/build inventory
Technology Discovery Agent	Identify languages, frameworks, runtimes, databases, integrations and versions.	Technology inventory
Dependency Analysis Agent	Analyze direct/transitive dependencies and target compatibility.	Dependency inventory and compatibility findings
Source Analysis Agent	Identify source-level constructs affected by the requested transformation.	Code compatibility findings
Framework Analysis Agent	Analyze framework usage and modernization impact.	Framework findings and affected components
Configuration Analysis Agent	Analyze configuration, build and deployment artifacts.	Configuration findings
Quality & Security Agent	Identify issues that can affect modernization or target readiness.	Quality/security findings

These agents should not independently make the final migration decision. Their findings are consolidated before feasibility is determined.

7. Assessment Consolidation and Feasibility

7.1 Assessment Aggregator

The Assessment Aggregator combines the outputs from discovery and analysis agents, removes duplicate observations, groups related findings and creates a single application assessment.

Finding Attribute	Purpose
Category	Technology, dependency, framework, source, configuration, security, etc.
Severity	Critical, high, medium or low based on defined rules.
Evidence	File/module/configuration reference supporting the finding.
Impact	Expected effect on the requested transformation.
Suggested Action	Potential remediation or further analysis.
Automation Potential	Whether the issue can be handled automatically, requires review, or requires architecture-level decision.

7.2 Feasibility Agent

The Feasibility Agent uses the consolidated evidence to determine the migration state:

Status	Meaning
Feasible	The requested change can proceed using the identified migration approach.
Conditionally Feasible	The change is achievable but one or more approved remediation/architecture changes are required.
Blocked	A current constraint prevents the requested transformation until a decision or prerequisite is resolved.

8. Alternatives and Human-in-the-Loop

When the requested target cannot be reached through a direct change, the platform should present alternatives rather than silently changing the target architecture.

Alternative Information	What the user should see
Approach	What will change and why.
Affected Areas	Modules, frameworks, dependencies or interfaces impacted.
Impact	Expected technical and application-level consequences.
Risks	Known migration or compatibility risks.
Validation Impact	Additional testing required.
Decision	Approve, reject or request reassessment.

Example

If an existing framework is incompatible with the desired runtime, the platform may present a framework upgrade, framework replacement, or a constrained compatibility approach. The user approves the strategy before execution begins.

9. Stage C — Migration Planning

Once the user approves the strategy, the Migration Planner converts the decision into an execution blueprint.

Planning Output	Description
Migration Phases	Logical sequence of modernization activities.
Tasks	Specific changes to be performed.
Dependencies	Order in which tasks must be executed.
Checkpoints	Build/test points after significant changes.
Rollback Strategy	How changes can be reverted or isolated.
Validation Plan	Checks required to confirm successful completion.

A typical generic sequence is:

1. Prepare isolated workspace and baseline.
2. Update build/runtime configuration.
3. Update or replace incompatible dependencies.
4. Apply source-level transformations.
5. Apply framework or technology transformations.
6. Update configuration and deployment artifacts.
7. Build and execute tests.
8. Resolve failures and repeat validation.
9. Run final validation and generate reports.

10. Stage D — Migration / Transformation Execution

Execution is performed by specialized agents selected according to the approved migration plan. The platform should not assume that every transformation requires the same agents.

Execution Agent Type	Typical Responsibility
Build Configuration Agent	Update compiler/runtime/build configuration.
Dependency Migration Agent	Upgrade, replace or remove dependencies.
Source Transformation Agent	Apply source-level changes required by the target.

Framework Transformation Agent	Apply approved framework-specific changes.
Configuration Migration Agent	Update application, server and deployment configuration.
Test Remediation Agent	Update tests where the approved transformation changes APIs or behavior.

The actual set of execution agents is selected from the migration plan. For a simple version upgrade, only a subset may be required; a technology switch may require several transformation agents.

11. Stage E — Validation and Recovery Loop

Validation is part of execution, not a final afterthought. Every significant migration phase should be followed by an appropriate check.

Check	Purpose
Build	Confirm that the transformed application compiles/packages successfully.
Unit Tests	Confirm component-level behavior.
Integration Tests	Confirm interaction between application components and dependencies.
Static Analysis	Identify new code-quality or security issues.
Runtime Validation	Confirm the application starts and key flows operate where an executable environment is available.
Regression Validation	Compare important application behavior before and after transformation where test coverage permits.

When a validation check fails, the workflow follows:

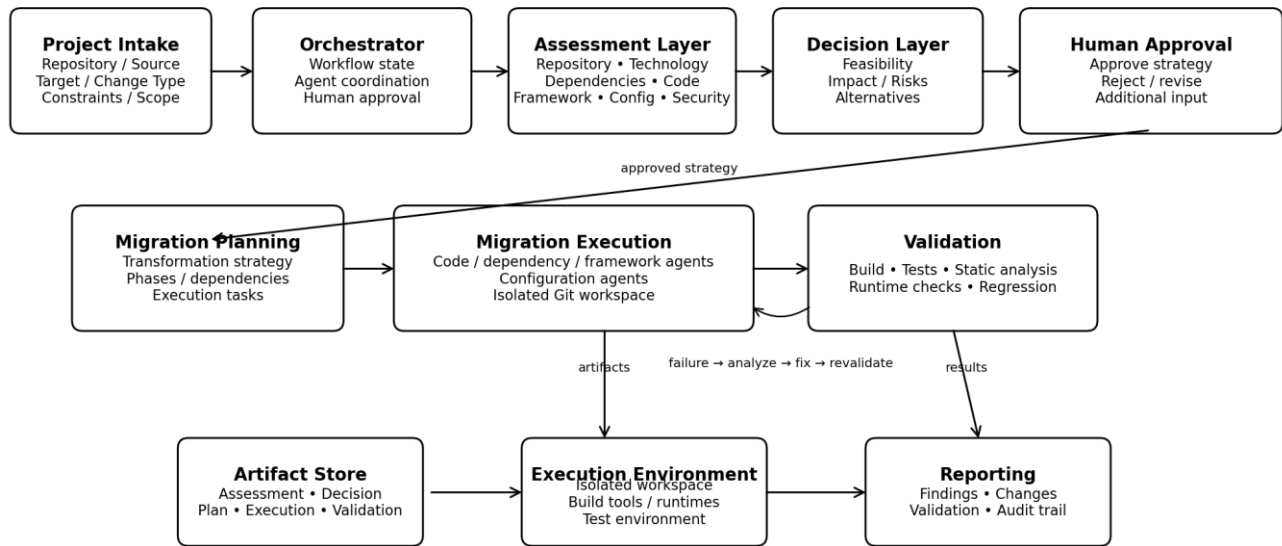
Failure → Failure Analysis → Remediation → Rebuild/Test → Continue or Escalate

12. Solution Architecture

The architecture separates user interaction, orchestration, intelligence, execution and evidence. This allows new modernization profiles to be added without redesigning the complete application.

Mira Modernize — Generic Application Modernization Architecture

Assess → Decide → Plan → Execute → Validate



Technology-agnostic orchestration; transformation profiles determine the source, target and applicable agents.

13. Logical Components

Component	Responsibility
Web Application	Project creation, assessment dashboard, findings, approvals, migration progress and reports.
API / Application Service	Exposes project, assessment, migration and reporting APIs.
Migration Orchestrator	Controls workflow, invokes agents, manages state and handles retries/escalation.
Agent Runtime	Executes assessment, planning, transformation and validation agents.
Repository Service	Clones/accesses repositories and manages branches/workspaces.
Execution Environment	Provides isolated runtimes, build tools, test environments and required SDKs.
Artifact Store	Persists structured artifacts and results for every stage.
AI/Model Gateway	Provides a pluggable interface to approved AI models.
Observability	Captures agent status, logs, prompts/outputs where permitted, build output and execution metrics.

14. Artifact-Driven Agent Communication

The platform should use structured artifacts as the contract between stages. Agents may use AI reasoning internally, but their externally visible output should be structured and machine-readable.

Artifact	Purpose
Modernization Request	Defines source, target, scope and constraints.
Repository Manifest	Defines repository structure and application

	boundaries.
Technology Inventory	Defines detected technologies and versions.
Dependency Inventory	Defines dependencies and compatibility information.
Assessment Findings	Defines evidence-based issues and impacts.
Assessment Report	Provides the consolidated modernization assessment.
Migration Decision	Stores the human-approved strategy.
Migration Blueprint	Defines phases, tasks, dependencies and checkpoints.
Execution Results	Records changes, commits, builds, tests and failures.
Validation Report	Provides final evidence and remaining issues.

15. Generic Usage Examples

The following examples illustrate the intended breadth of the platform. They are examples of request types, not hard-coded product behavior.

Scenario	Source	Target / Change	Expected Platform Behavior
Runtime upgrade	Java 8 application	Java 17	Assess APIs/dependencies/frameworks, plan changes, execute approved transformations and validate.
Framework modernization	Legacy Java web framework	Modern supported framework	Identify framework usage, map affected components, present alternatives, then execute the approved framework migration.
Frontend migration	Legacy frontend framework	Modern frontend framework	Analyze components, routing, dependencies and build system; generate migration plan and execute supported transformations.
Runtime/library upgrade	Existing Node.js application	New Node.js version + dependency upgrades	Assess runtime and package compatibility, update dependencies/source/configuration, then validate.
Technology switch	Application using Technology A	Technology B	Analyze architecture and dependencies, identify transformation scope, obtain approval, then execute transformation tasks.
Application enhancement	Existing application	Existing technology + new capability	Understand current architecture, identify affected components, plan implementation, execute changes and validate regression impact.

16. Extensibility Model

Generic behavior should live in the orchestration platform. Technology-specific behavior should be isolated in modernization profiles.

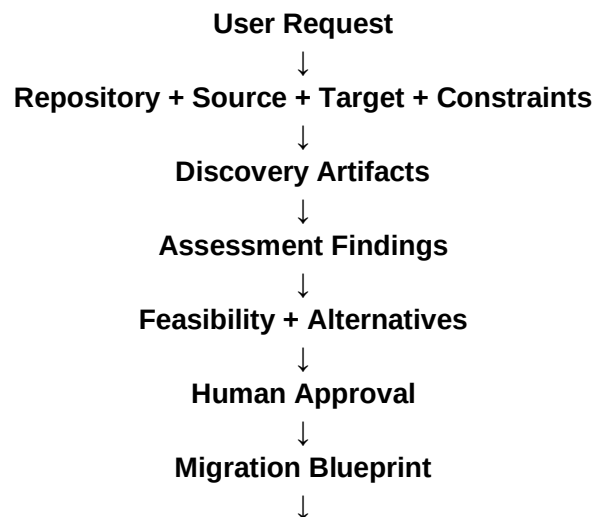
Layer	Generic or Specific?	Examples
Project / workflow management	Generic	Projects, approvals, state, audit trail
Repository discovery	Mostly generic	Git providers, file inventory,

		module detection
Assessment framework	Generic	Finding model, severity, evidence, reporting
Transformation profile	Technology-specific	Java 8→17, framework A→B, Node runtime upgrade
Execution agent	Technology-specific	Java source transformer, Angular transformer, dependency upgrader
Validation framework	Generic	Build/test/result collection
Reporting	Generic	Assessment, migration and validation reports

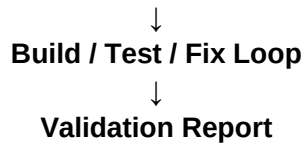
17. Key Non-Functional Requirements

Requirement	Expectation
Security	Repository access and source code must be isolated and protected. Credentials must not be embedded in prompts or logs.
Traceability	Every important finding and code change should be traceable to a project, stage and agent execution.
Isolation	Code transformation must occur in an isolated workspace/environment.
Recoverability	A failed agent or phase should be restartable without losing completed artifacts.
Extensibility	New modernization profiles and agents should be addable without changing core orchestration.
Observability	Users should be able to see current stage, progress, failures and validation status.
Human Control	Architecture-changing alternatives require explicit approval before execution.
Model Independence	AI model/provider should be replaceable through an abstraction layer.
Auditability	Store decisions, approvals, execution results and validation evidence.

18. Simplified Data Flow



Execution Workspace + Transformation Agents



19. Implementation Guidance for Engineering Team

The first implementation should establish the generic orchestration framework and one complete transformation profile. The profile should be sufficiently representative to prove assessment, approval, execution and validation end to end.

- Implement the project/request model and workflow state first.
- Define JSON schemas/contracts for the major artifacts before implementing individual agents.
- Implement repository discovery and technology/dependency inventory as reusable services.
- Build assessment agents around evidence extraction rather than asking an LLM to infer the entire repository from raw source.
- Implement the human approval gate before enabling automated transformation.
- Use isolated Git branches/workspaces for every migration execution.
- Make build/test feedback available to execution agents so migration is iterative.
- Keep transformation-specific rules outside the core orchestration code.
- Persist every stage so a failed workflow can resume from the latest successful checkpoint.

20. Expected Product Outputs

Output	Audience / Purpose
Assessment Report	Architects and engineering teams; explains current state, impact and feasibility.
Migration Decision	Records the approved target approach and human approval.
Migration Plan	Implementation team; defines phases, tasks and dependencies.
Execution Report	Engineering team; summarizes automated changes and execution results.
Validation Report	Engineering/client stakeholders; provides build, test and quality evidence.
Audit Trail	Engineering/governance; records decisions, agent executions and checkpoints.

21. Recommended Terminology

To keep communication consistent across Innovation Labs and delivery teams, use the following terminology:

Term	Use
Modernization Request	The user's requested application change.
Assessment	The analysis of the current application against the requested target.
Finding	An evidence-backed issue or impact identified during assessment.
Migration Strategy	The approved approach for achieving the target

	state.
Migration Plan	The ordered set of phases and tasks implementing the strategy.
Transformation Agent	An agent responsible for a defined category of code/configuration change.
Validation	The checks performed to establish whether the target state was achieved.
Escalation	A controlled pause requiring additional human input.

22. Conclusion

Mira Modernize should be implemented as a generic modernization orchestration platform rather than as a Java upgrade utility. The platform should understand a modernization request, analyze the application, identify the impact of the requested change, present evidence and alternatives, obtain explicit approval for significant decisions, execute the approved plan through specialized agents, and validate the resulting application.

The architecture therefore separates generic workflow capabilities from technology-specific modernization profiles. This allows the same platform to support runtime upgrades, framework upgrades, dependency modernization, technology switches and controlled application enhancements without redesigning the core system.